Table 'alertes' : rôle et structure

La table `alertes` est le cœur du suivi des anomalies détectées par le moteur. Chaque ligne correspond à une alerte générée suite à l'analyse d'une transaction.

Champs principaux :

Champ	Type	Description
id	UUID	Identifiant unique de l'alerte
transaction_id	UUID	Identifiant de la transaction liée
code	Texte	Code d'alerte (ex : ALERT001, ALERT004B…)
type	Texte	Type d'alerte (Montant, Blacklist, PEP, etc.)
statut	Texte	Statut métier (En cours, Traité, Faux positif…)
message	Texte	Message explicite pour l'analyste ou l'intégration
niveau_criticite	Texte	Criticité (Haute, Moyenne, Faible)
score_risque	Décimal	Score de risque calculé
analyste_assigne_id	UUID	Identifiant de l'analyste assigné (optionnel)
date_generation	DateTime	Date et heure de création de l'alerte

Usage métier :

- Permet à l'analyste de prioriser et traiter les alertes selon leur criticité et leur type.
- Historise toutes les anomalies détectées pour audit et reporting.
- Sert de base à l'interface de supervision et à l'API d'alertes.

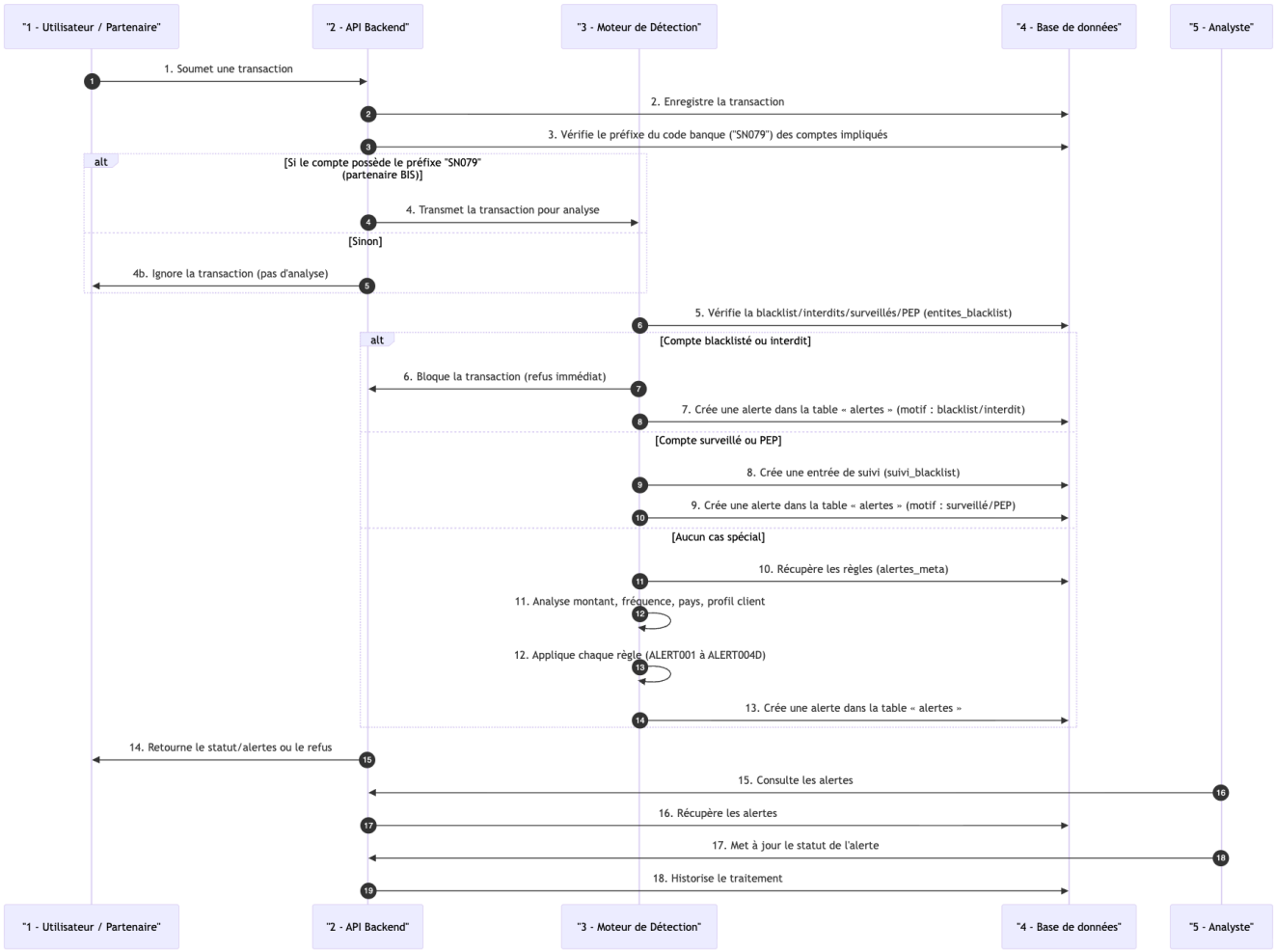
Lien avec les règles :

- Les champs `code`, `type`, `statut`, `message`, `niveau_criticite` sont alimentés dynamiquement selon la table `alertes_meta`.

Exemple d'enregistrement :

id	transaction_id	code	type	statut	message	niveau_criticite	score_risque	analyste_assigne_id	date_generation
7e2a…	1a3b…	ALERT004B	Blacklist	En cours	Compte blacklisté : transaction refusée.	Haute	0.95		2025-10-15T10:00:00Z

Diagramme de séquence du moteur de détection d'anomalies



Explication du diagramme de séquence

1. **Soumission de la transaction** : L'utilisateur ou le partenaire envoie une transaction à analyser via l'API.
2. **Enregistrement** : L'API sauvegarde la transaction dans la base de données.
3. **Vérification du code banque** : L'API vérifie si l'un des comptes impliqués possède le préfixe "SN079" (partenaire BIS).
4. **Transmission ou Ignoré** :

- Si le compte appartient au partenaire BIS, la transaction est transmise au moteur de détection pour analyse.
- Sinon, la transaction est ignorée et aucune analyse n'est faite.

5. **Vérification blacklist/interdits/surveillés/PEP** : Le moteur vérifie si le compte est présent dans la table `entites_blacklist`.

- Si le compte est blacklisté ou interdit, la transaction est immédiatement bloquée et une alerte est créée (motif : blacklist/interdit). Les autres règles ne sont pas appliquées.
- Si le compte est surveillé ou PEP, une entrée de suivi est créée dans `suivi_blacklist` et une alerte est générée (motif : surveillé/PEP). Les autres règles ne sont pas appliquées.
- Si aucun cas spécial n'est détecté, le moteur poursuit l'analyse métier.

6. **Application des règles métier** :

- Le moteur récupère dynamiquement les règles d'alerte (`alertes_meta`).
- Il analyse les critères : montant, fréquence, pays, profil client, etc.
- Il applique chaque règle métier (ALERT001 à ALERT004D).
- Si une anomalie est détectée, une alerte est créée dans la table `alertes`.

7. **Retour API** : L'API retourne le statut et les alertes générées à l'utilisateur ou au partenaire.
8. **Consultation analyste** : L'analyste consulte les alertes via l'interface ou l'API.
9. **Récupération** : L'API récupère les alertes depuis la base pour affichage ou traitement.
10. **Mise à jour** : L'analyste met à jour le statut de l'alerte (traitée, faux positif, etc.).
11. **Historisation** : L'API historise le traitement pour audit et reporting.

Documentation des Alertes – API Anomaly Detector

---

1. Structure d'une alerte

```
{
  "id": "string",
  "transactionId": "string",
  "code": "string",
  "type": "string",
  "statut": "string",
  "message": "string",
  "niveauCriticite": "string",
  "scoreRisque": "float",
  "analysteAssigneID": "string",
  "dateGeneration": "datetime"
}
```

2. Règles d'alerte utilisées dans le backend

Les règles d'alerte sont définies dans la table `alertes_meta` et utilisées pour la détection d'anomalies. Voici la liste des règles actuellement en production :

Code	Type	Statut	Message	Criticité
ALERT001	Montant	En cours	Cumul sur 7 jours dépassant le seuil autorisé.	Haute
ALERT002	frequence	En cours	Nombre de transactions sur 24h dépassant le seuil autorisé.	Moyenne
ALERT003	Pays à risque	En cours	Transaction vers ou depuis un pays à risque.	Haute
ALERT004A	Interdits	En cours	Compte interdit : transaction refusée.	Haute
ALERT004B	Blacklist	En cours	Compte blacklisté : transaction refusée.	Haute
ALERT004C	PEP	En cours	Personne politiquement exposée : transaction suivie.	Haute
ALERT004D	Surveillés	En cours	Compte sous surveillance : transaction suivie.	Moyenne

Chaque règle possède :

- Un code unique (`code`)
- Un type d'alerte (`type`)
- Un statut (`statut`)
- Un message explicite (`message`)
- Un niveau de criticité (`niveau_criticite`)

Ces valeurs sont utilisées pour générer et structurer les alertes dans le backend et l'API.

Autres champs

- **Score de risque (scoreRisque)** : Nombre décimal calculé selon la règle déclenchée.
- **Analyste assigné (analysteAssigneID)** : Identifiant de l'analyste (optionnel, peut être vide si non assigné).
- **Date de génération (dateGeneration)** : Date et heure de création de l'alerte.

---

3. Exemples d'alerte

```
{
  "id": "a1b2c3",
  "transactionId": "t123",
  "code": "ALERT001",
  "type": "Montant",
  "statut": "En cours",
  "message": "Transaction de montant élevé détectée",
  "niveauCriticite": "Haute",
  "scoreRisque": 0.92,
  "analysteAssigneID": "",
  "dateGeneration": "2025-10-15T10:00:00Z"
}
```

---

4. Utilisation dans l'API

- Les alertes sont générées automatiquement lors de l'analyse des transactions.
- Elles sont accessibles via l'endpoint : `GET /api/v1/alertes`
- Pour chaque alerte, les champs ci-dessus sont renseignés.

- Les valeurs de code, type, statut, message et criticité sont dynamiques et peuvent être enrichies via la table `alertes_meta`.

---

5. Bonnes pratiques pour l'analyste

- Se référer au champ `code` pour identifier rapidement la nature de l'alerte.
- Utiliser le champ `message` pour comprendre le contexte métier.
- Prioriser le traitement selon le champ `niveauCriticite`.
- Utiliser le statut pour suivre l'avancement (En cours, Traité, Faux positif).
- Consulter l'historique via l'API pour les alertes déjà traitées.

Pour toute question ou besoin d'exemple supplémentaire, se référer à la table `alertes_meta` ou à l'équipe technique.